

REVISION CONTROL

Nick Brown

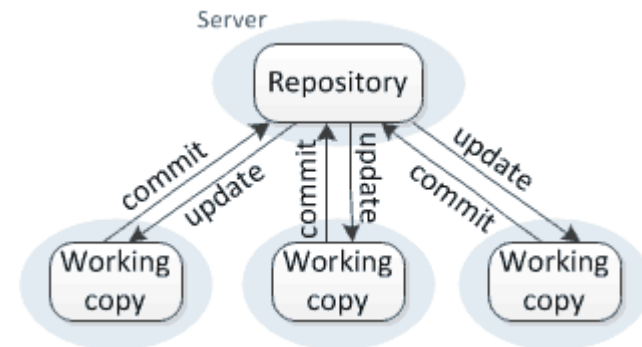
Room 3419

nick.brown@ed.ac.uk



What is revision control?

- Revision control is an aspect of code management
 - But it is a very important aspect
- Revision control manages your files in a variety of ways
 - It records changes over time in a retrievable manner
 - Developers can (and should) add comments to each version
 - This results in a documented history of the source code
- Uses a code “repository”
 - A safe storage area away from day-to-day coding
 - The repository is “public”; working copies are “private”



Revision control does

- Centralised storage of project files (single point of truth)
- Retrievable versions
- Clear history of changes

-
- Allows multiple developers to work on the same files
 - Supports branching and merging of files to support good Software Engineering

Revision control and working together

- Supports multiple users
 - Work on the same files
 - Merge changes from two users
 - Resolve conflicts
 - Record who, what, when and why
- Is a critical aspect of many software engineering teams

Why use revision control?

- Backup – it's a safe copy of your code
 - perhaps you've accidentally deleted your code
 - or made a large change, and want to 'rewind' to a previous version
 - ***Like an “undo” for coding***
- Keep track of what you're doing
 - whenever you make a change you can review it
 - changes can be documented
- Share all of this with others
 - a whole team can work on the same code
 - (different systems handle this differently, as we will see)
 - each developer can see everyone's documented changes
- What's the alternative?

Revision control and science

- Provenance for software
 - What was changed
 - Who changed it
 - When they changed it
 - Why they changed it
- Provenance for research
 - History of your research ideas
 - Associate software versions with data and papers

*Acts as a log book for software including your ideas,
evolution, fixed and false paths*

Provenance

```
RCS file: /var/cvs/ogsa/ogsa-dai-
cvs/source/core/src/resource/uk/org/ogsadai/resource/dataresource/jdbc/JDBCDataResource.java,v
Working file: source/core/src/resource/uk/org/ogsadai/resource/dataresource/jdbc/JDBCDataResource.java
head: 1.17
```

```
branch:
```

```
locks: strict
```

```
access list:
```

```
symbolic names:
```

```
total revisions: 17;    selected revisions: 17
```

```
description:
```

```
-----
revision 1.17
```

```
date: 2008/11/26 13:39:12; author: michaelj; state: Exp; lines: +15 -1
```

```
MJ: OGSA-DAI-Licence.txt replaced with Apache 2.0
-----
```

```
revision 1.16
```

```
date: 2007/08/17 17:56:52; author: michaelj; state: Exp; lines: +1 -1
```

```
Fixed copyright comments and statements to read The University of Edinburgh
-----
```

```
revision 1.15
```

```
date: 2007/08/07 13:03:26; author: michaelj; state: Exp; lines: +2 -6
```

```
DataResourceAuthorization deprecated. Data resource connection providers no longer throw authorization
exceptions so activities no longer need to catch these. Any problems in using a login provider are
bundled in an server-side JDBCConnection/XMLDBCollection/FileResourceUse exception
-----
```

```
revision 1.14
```

```
date: 2007/07/26 15:26:01; author: amrey; state: Exp; lines: +23 -11
```

```
Merged interface XXX and implementing class SimpleXXX for ID, ResourceID, ActivityName,
ActivityInstanceName, UpdateID, ServerOpName.
```

When

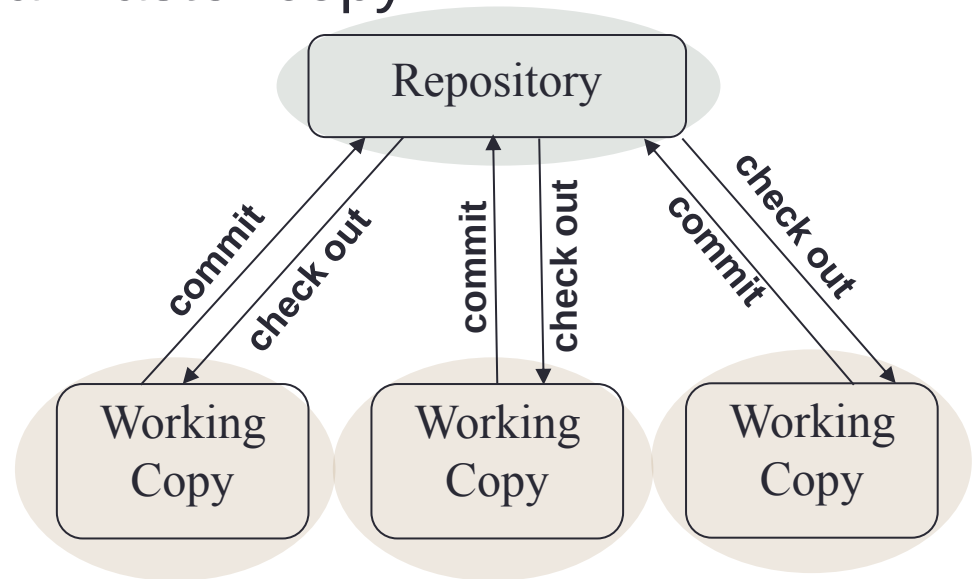
Who

What

Why

Revision control concepts

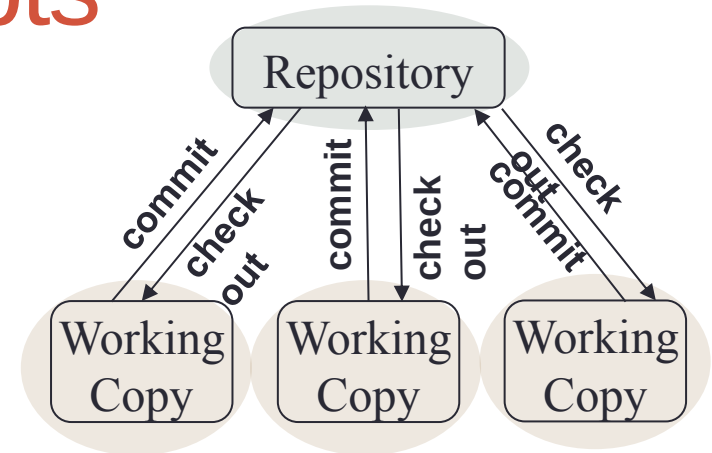
- Working (or local) copy is what you work with
- Repository (or master) is a master copy
 - Directories
 - Source code
 - Configuration files
 - Documents



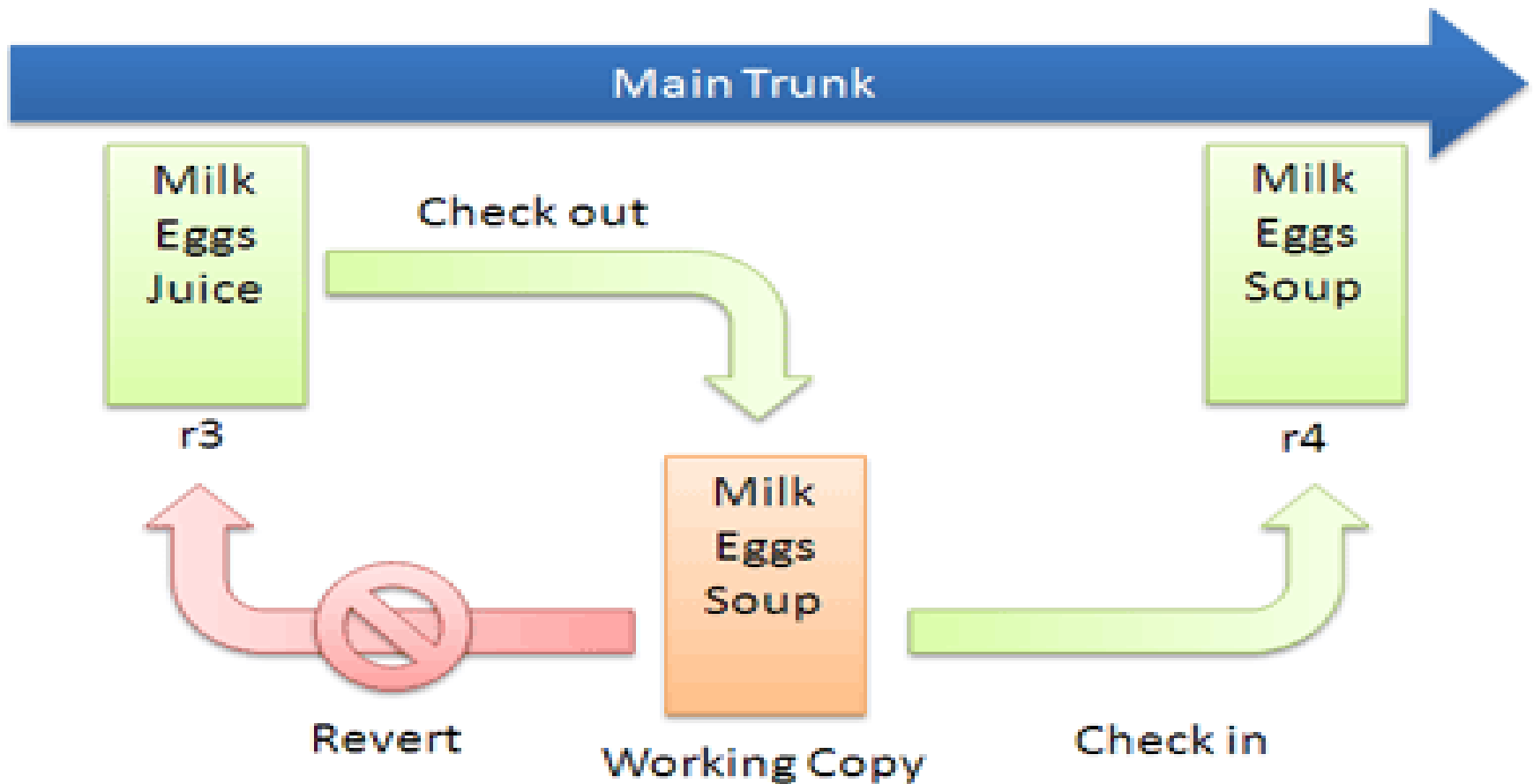
- You can retrieve changes from any point in time
 - “Oh, that way I did it before was better after all”

Revision control concepts

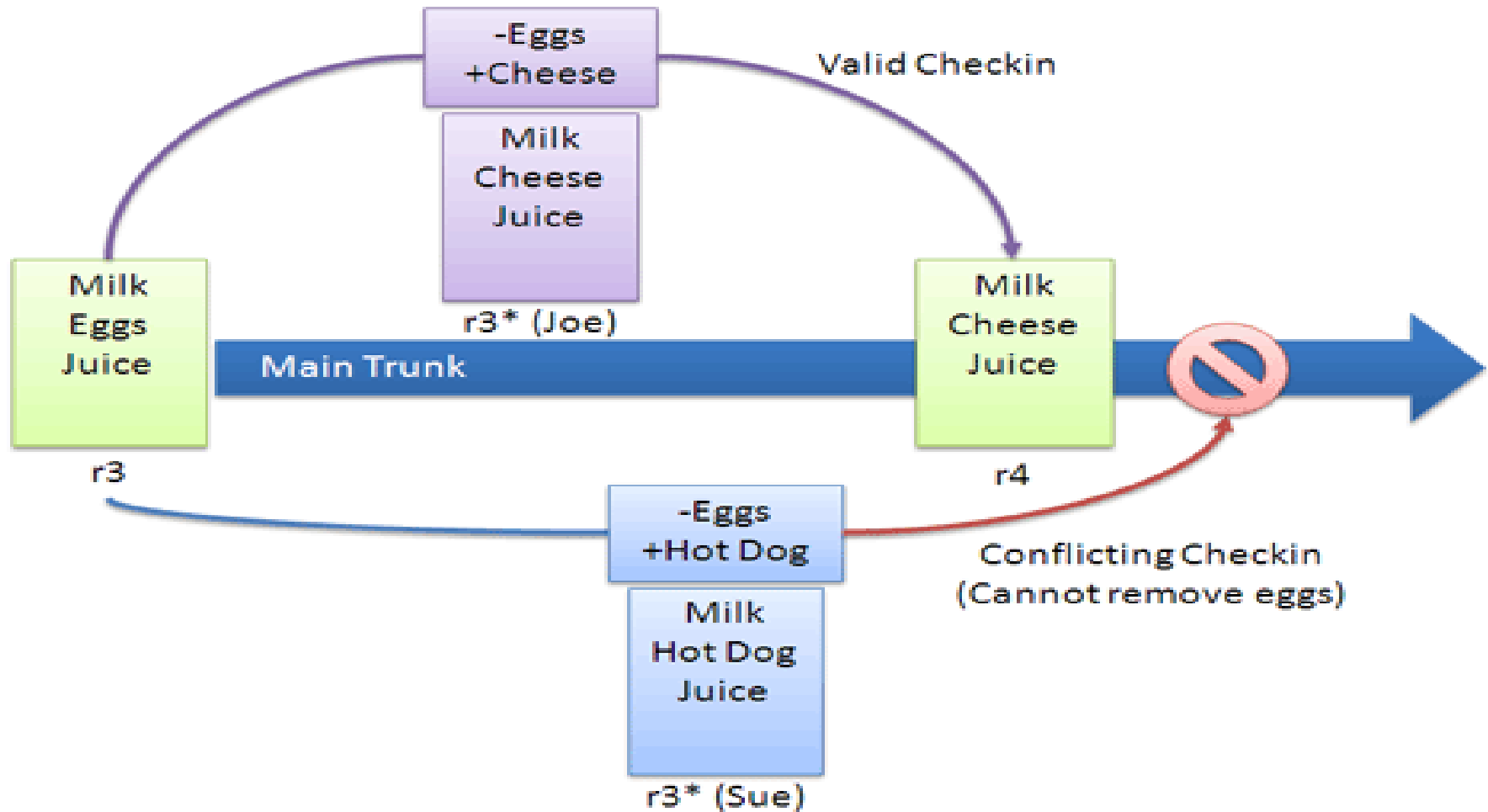
- Working code is “checked out”
 - The developer gets a local copy of the code to work on
 - this copy is special, as it represents the code at a specific point in time
 - The developer can then work on this checked-out version
 - Checked out versions are “private” and “local”
- Revisions to be saved are “checked in” or “committed”
 - Checked in versions are saved in the Repository
 - if the Repository is shared, this “publishes” the changes so that others can access them
 - These changes can be commented upon and documented as they are checked in via a commit message



Checkout and Edit



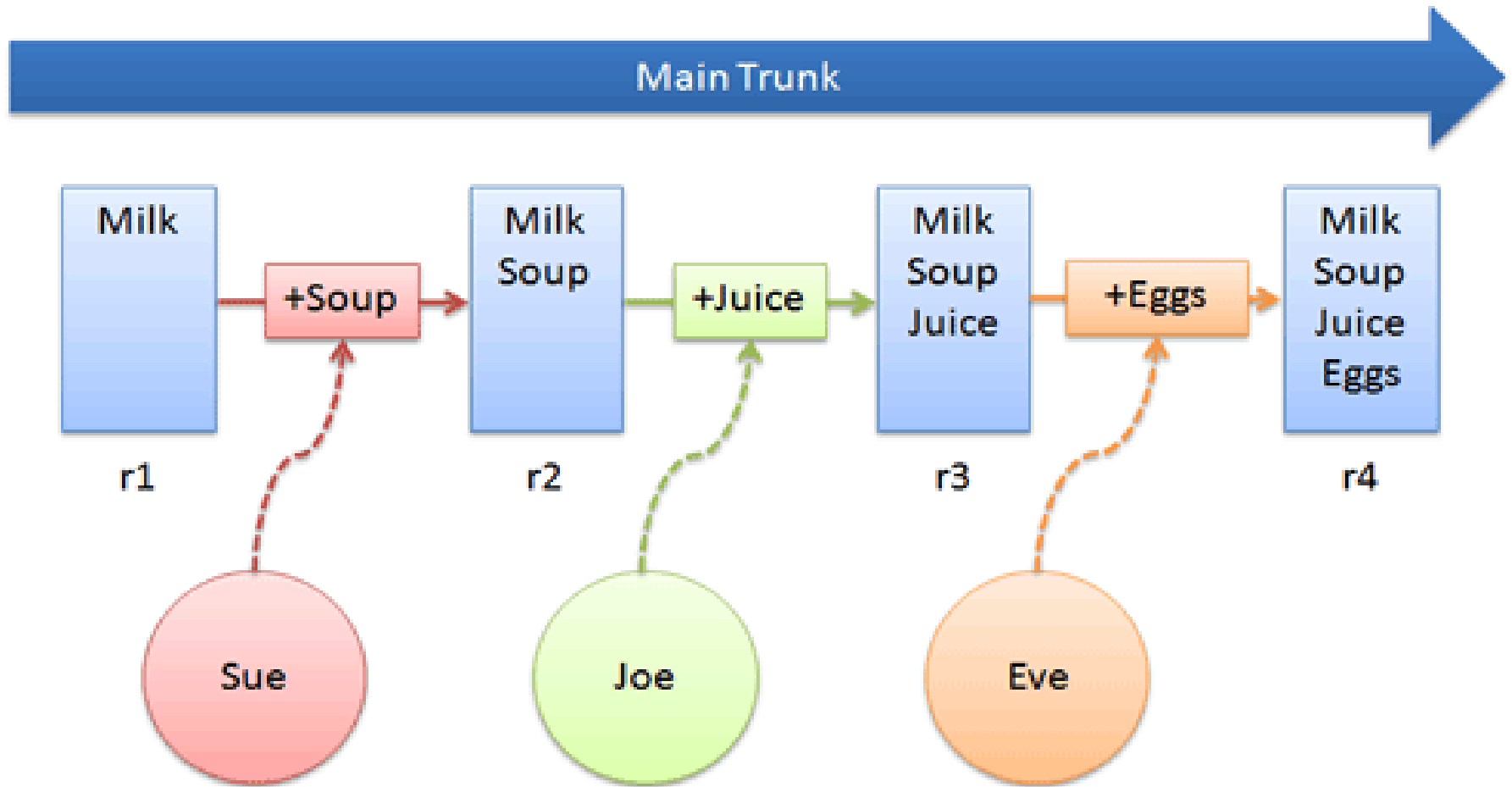
Conflicts



Versioning Models

- Problem of file sharing
 - Want to enable multiple users to develop same code base
 - Need to avoid overwriting of other people's changes, even if history preserved this is a problem
- Lock-modify-unlock
 - Basic control system, uses locks for files or directories
 - Only one person allowed to modify a file at any one time
 - Restrictive to development
- Copy-modify-merge
 - Common solution, allows concurrent development
 - Differences in files addressed at repository when update performed
 - Conflicts between public and private versions resolved by user
 - Not applicable for some file types (i.e. images)

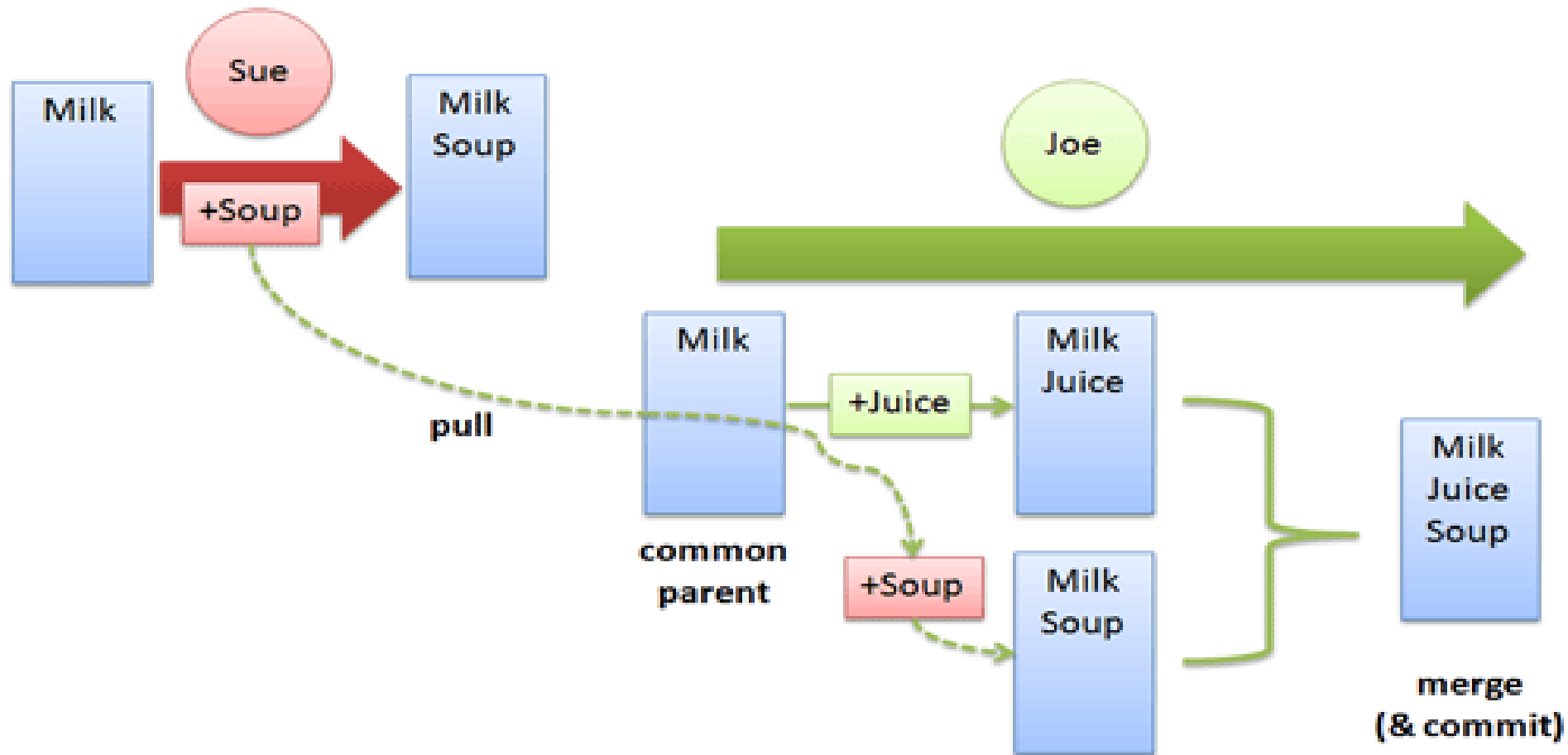
Lock – modify - unlock



Versioning Models

- Problem of file sharing
 - Want to enable multiple users to develop same code base
 - Need to avoid overwriting of other people's changes, even if history preserved this is a problem
- Lock-modify-unlock
 - Basic control system, uses locks for files or directories
 - Only one person allowed to modify a file at any one time
 - Restrictive to development
- Copy-modify-merge
 - Common solution, allows concurrent development
 - Differences in files addressed at repository when commit performed
 - Conflicts between public and private versions resolved by user
 - Not applicable for some file types (i.e. images)

Copy – modify - merge



Advanced revision control concepts

- Branching and Merging

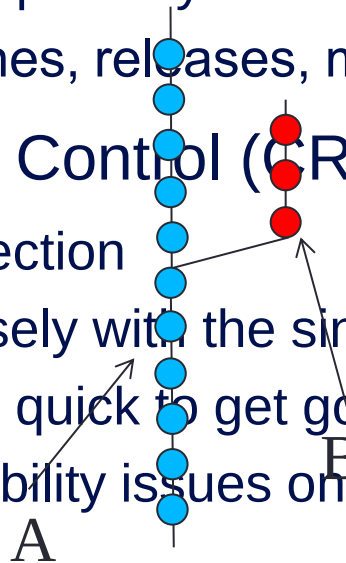
- A Branch is a new version of an item, collection or project. It can be thought of as a copy that can be changed independently
- Merging is process of taking changes from branches and applying them to the main branch and/or vice versa

- Tag, Baseline, Label

- Snapshot of source/repository
- Often used for branches, releases, milestones, etc...

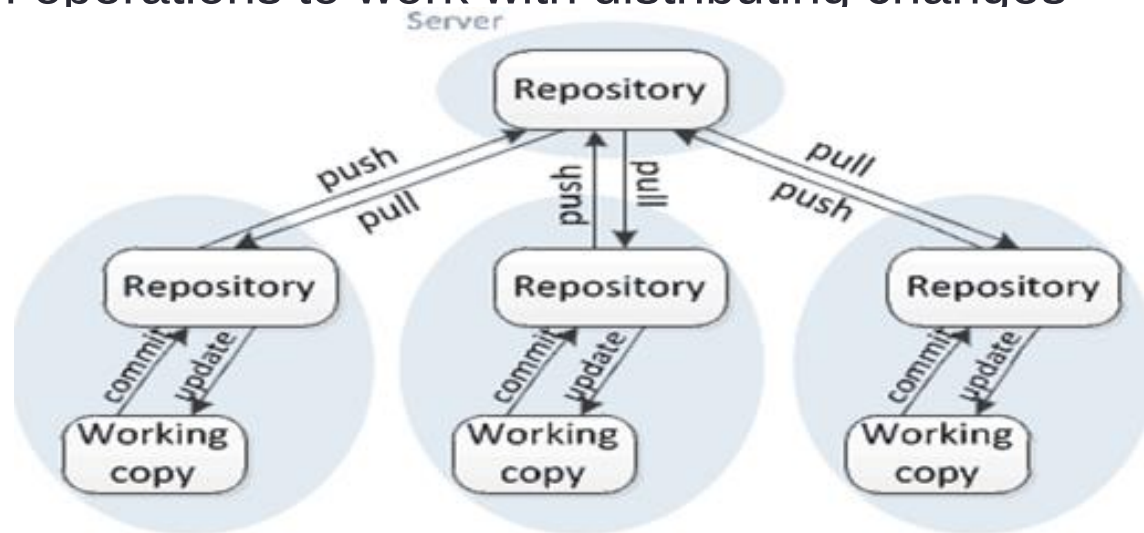
- Centralised Revision Control (CRCS)

- Client to server connection
- Clients work very closely with the single repository
- A simple concept and quick to get going
- Can suffer from scalability issues on large projects with many contributors



Distributed Revision Control

- Distributed Revision Control (DRCS)
 - Peer-to-Peer rather than Client-Server
 - Multiple/many repositories
 - Many operations quicker than client/server as performed locally
 - New set of operations to work with distributing changes



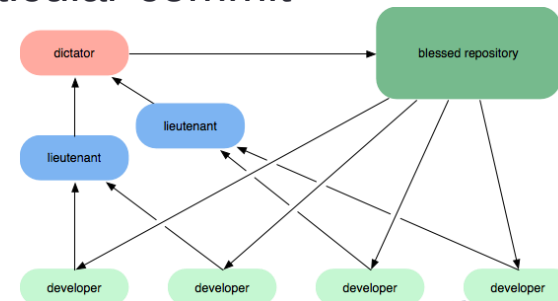
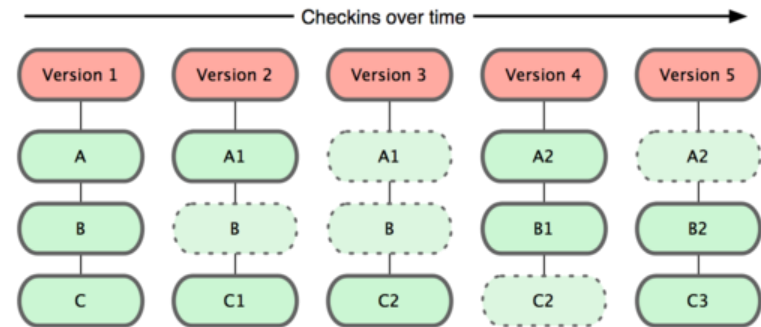
- Enables massive scalability
 - Not only via the architecture but also more advanced support for areas such as branching

Revision control scenarios

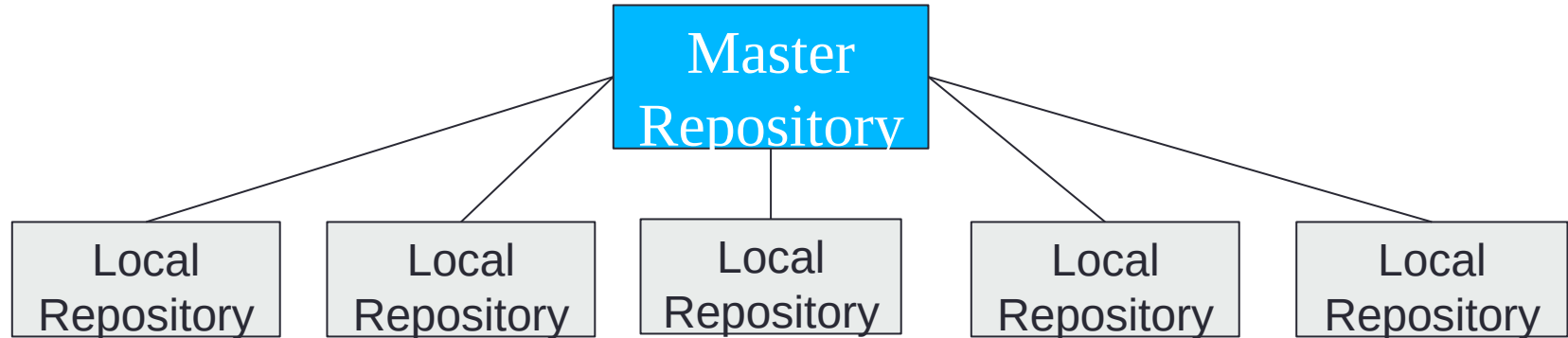
- You accidentally delete your working copy of the code
 - Just check out another copy!
- A bug is found in your software a year after release
 - Since revision control stores the entire history of the code, it is possible to test previous versions
- You want to use a function that you deleted yesterday
 - Just check out yesterday's version, and copy it across
- You are working on multiple machines
 - Your repository acts as a centralised storage of files which can be accessed from multiple locations

Git

- Distributed revision control system
 - Good for large, distributed, development projects
 - Each participant has full working copy
 - Changes copied between working copies, merged as local branches
 - Fast and scalable
- Snapshots directory trees of files
 - Not based on diff in files
 - More storage but quicker
- Staged commit
 - Staging area allows modifications to be chosen for a particular commit
- Flexible, local branching
 - All branching local unless pushed
 - Multiple, independent, branches allowed
- Different revision control structures easily achievable



Git Basic Concepts



- Each repository has a (hidden) `.git` directory
 - Holds the history/change information and configuration
 - Also stores “pristine” copies of files
- Two stage push model
 - First commit your changes from the working copy into your local repository
 - Secondly push these committed changes from your local repository into the master repository

```
$ git SUBCOMMAND OPTIONS ARGS  
    add, bisect, branch, checkout, clone, commit,  
    diff, fetch, grep, init, log, merge, mv, pull,  
    push, rebase, reset, rm, show, status, tag
```

Why Git?

- Advantages of Git
 - Always “online” with respect to local repository
 - Designed to be very fast
 - Branches are lightweight
 - Merging is simpler, more automatic and integrated with file history
 - File format excellent at compressing data
 - I can work how I want, the master owner can not dictate how I work
- Git weak points
 - Steeper learning curve

Git has been designed so that many of the commands bear a resemblance to SVN commands to ease transition between tools

Revision Control Hosting

- GitHub
 - Free for open source software
 - Private costs money and is aimed at commercial development
 - Subversion and Git
- Bitbucket
 - Free hosting for private repositories with up to 4 users
 - Git and Mercurial
- Sourceforge
 - Git, Subversion, CVS
 - Open source projects